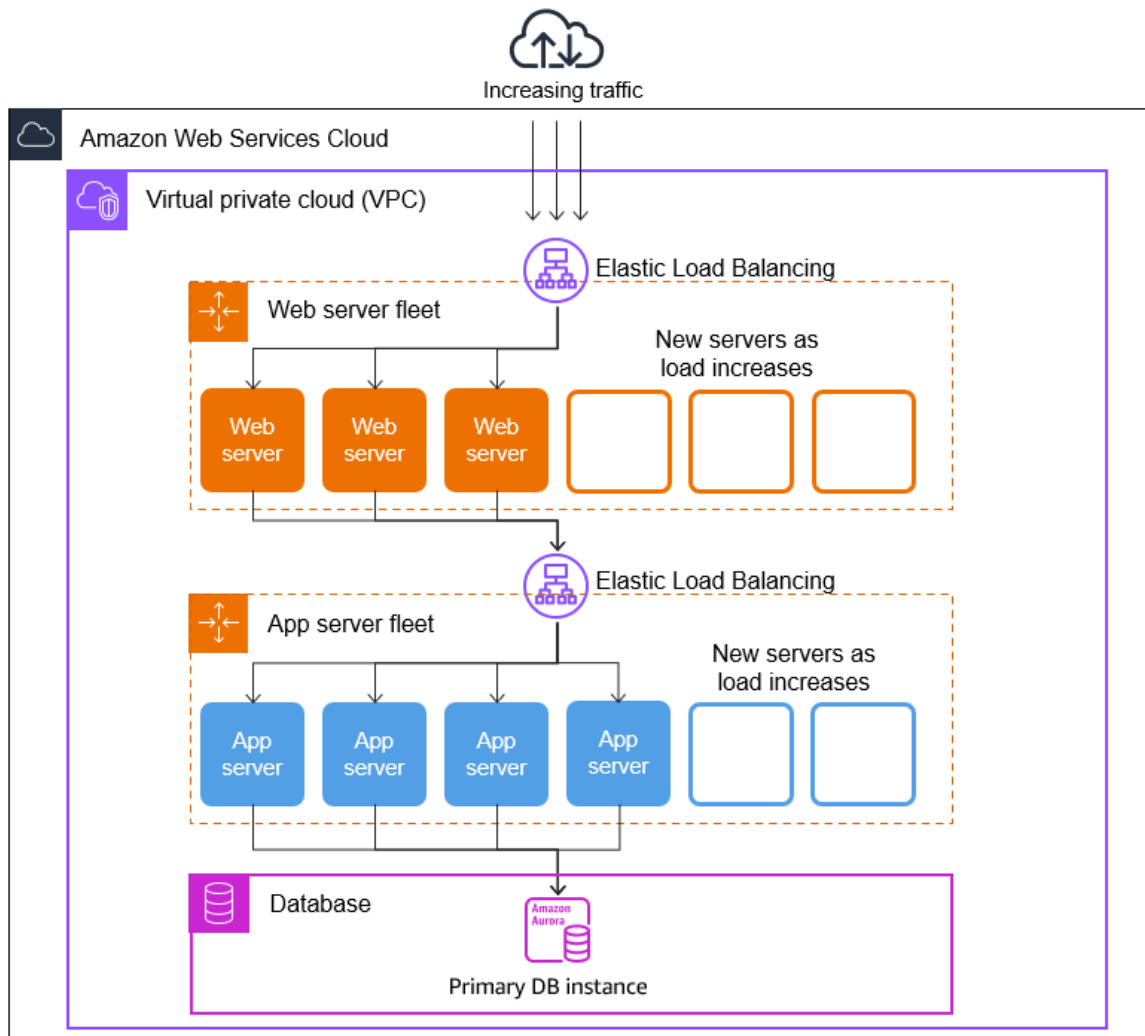


1.2 Características.

1.2.1 Escalabilidad y Elasticidad

Concepto: Capacidad para ajustar los recursos de manera automática según la demanda del usuario.

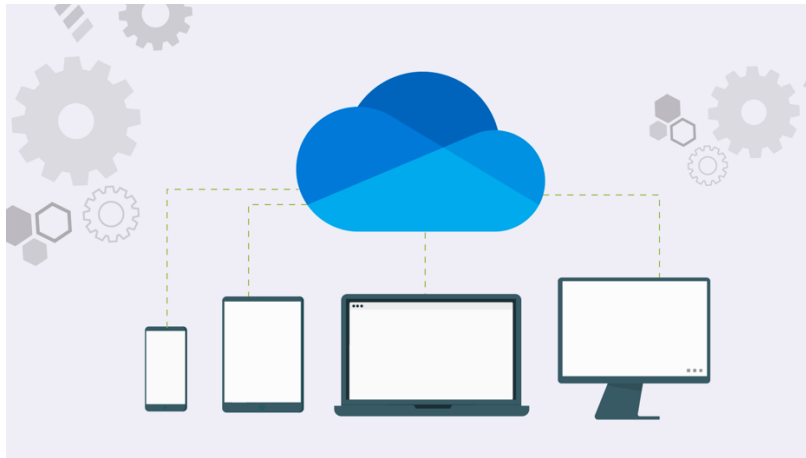
Ejemplo: Amazon Web Services (AWS) ofrece **Auto Scaling**, que permite agregar más instancias de servidores cuando aumenta el tráfico a una aplicación web y reducirlas cuando el tráfico disminuye, optimizando costos y rendimiento.



1.2.2. Accesibilidad Remota

Concepto: Acceso a recursos desde cualquier lugar con conexión a Internet.

Ejemplo: Los servicios de Google Drive y Microsoft OneDrive permiten a los usuarios almacenar archivos en la nube y acceder a ellos desde cualquier dispositivo conectado a Internet, sin importar su ubicación física.



1.2.3. Modelo de Pago por Uso

Concepto: Facturación basada en el uso real de los recursos.

Ejemplo: AWS Lambda cobra solo por las funciones que se ejecutan y el tiempo que tardan en completarse, eliminando costos fijos por servidores.

AWS Lambda es un servicio de computación sin servidor (serverless) proporcionado por Amazon Web Services. Permite ejecutar código en respuesta a eventos, sin necesidad de aprovisionar o administrar servidores.

- “Sin servidor” significa que no tienes que preocuparte por la infraestructura: AWS se encarga de asignar los recursos necesarios automáticamente.
- Ejecuta funciones escritas en varios lenguajes como Python, Node.js, Java, C#, entre otros.

AWS Lambda se emplea para:

- Automatizar procesos (por ejemplo, ejecutar código cuando se sube un archivo a un bucket de S3).

Un bucket de S3 es un contenedor de objetos (archivos) en Amazon S3, el servicio de almacenamiento de objetos en la nube de AWS. En otras palabras, es el lugar donde se almacenan los datos en S3. Cada bucket tiene un nombre único a nivel global dentro de AWS y se encuentra en una región específica.
- Responder a eventos en tiempo real (como mensajes de una cola de Amazon SQS o cambios en una base de datos DynamoDB).
- Procesamiento backend de aplicaciones web y móviles.
- Microservicios: cada función Lambda puede representar un microservicio autónomo.
- Integración con otros servicios de AWS, como API Gateway, DynamoDB, S3, etc.

Lo utilizan:

- Desarrolladores y empresas tecnológicas que requieren soluciones escalables, automáticas y de bajo mantenimiento.
- Startups y pymes por su modelo de pago por uso.
- Corporaciones grandes para construir microservicios y arquitecturas orientadas a eventos.
- Instituciones educativas y desarrolladores independientes para prototipos y prácticas en la nube.

Principales funciones o características

Función / Característica	Descripción
Sin servidor	No se gestiona infraestructura: solo se sube el código.
Escalado automático	Se ajusta automáticamente a la demanda.
Alta disponibilidad	AWS gestiona múltiples zonas de disponibilidad.
Integración con AWS	Se conecta directamente con servicios como S3, DynamoDB, API Gateway.
Diversos lenguajes	Soporta Python, Node.js, Java, C#, Go, y más.
Facturación por uso	Solo pagas por el tiempo de ejecución del código (milisegundos).

AWS Lambda tiene un nivel gratuito y luego cobra según uso:

- El nivel gratuito de AWS ofrece 12 meses de prueba gratuita y servicios siempre gratuitos que no requieren pago inicial. Para Lambda, esto incluye un millón de solicitudes gratuitas y 400,000 GB-segundos de tiempo de cómputo al mes. Para S3, el nivel gratuito ofrece 5 GB de almacenamiento, 20,000 solicitudes GET y 2,000 solicitudes PUT al mes, además de 100 GB de transferencia de datos de salida al mes, según Amazon Web Services.

Lambda es un servicio de computación sin servidor que ejecuta código en respuesta a eventos, mientras que S3 es un servicio de almacenamiento de objetos en la nube. Lambda procesa datos y ejecuta lógica, mientras que S3 almacena archivos y datos de manera escalable.

- Precios (aproximados):
 - \$0.20 USD por 1 millón de solicitudes adicionales
 - \$0.00001667 USD por GB-segundo de uso de memoria (ajustable entre 128 MB y 10,240 MB)

Ejemplo:

Una función de 512 MB que se ejecuta por 1 segundo = $0.5 \text{ GB} * 1 \text{ segundo}$
= 0.000008335 USD por ejecución (más las solicitudes).

1.2.4. Multitenencia y Aislamiento de Recursos

Multitenencia (multitenancy) es un modelo de arquitectura en la nube donde múltiples usuarios o “inquilinos” (tenants) comparten los mismos recursos físicos (servidores, almacenamiento, red), pero sus datos y configuraciones están aislados entre sí.

En el contexto de Azure:

- Un tenant es una instancia dedicada de Azure Active Directory (Azure AD) para una organización.

- Varias organizaciones usan la misma infraestructura física de Microsoft Azure, pero cada una tiene sus propios servicios, datos y configuraciones lógicamente separados.
- El aislamiento de recursos garantiza que los datos, servicios y configuraciones de un tenant no interfieran con los de otro, incluso si usan la misma infraestructura subyacente.

Azure ofrece aislamiento en varios niveles:

Nivel de Aislamiento	Ejemplo / Herramienta Azure
Identidad	Azure Active Directory mantiene identidades únicas por tenant.
Red	Redes virtuales (VNet) privadas y controladas por firewalls y NSG.
Recursos	Grupos de recursos (Resource Groups) y políticas (Policies).
Contenedores / Apps	Azure Kubernetes Service (AKS), Azure App Service con contenedores.
Máquinas Virtuales	Las VM no comparten espacio con otras organizaciones, se aíslan con Hyper-V.

¿Para qué sirven estos conceptos?

- Seguridad y privacidad: Evitan fugas de datos entre clientes.
- Escalabilidad: Permiten a Azure manejar miles de organizaciones en la misma plataforma.
- Eficiencia: Ahorro de costos al compartir infraestructura sin comprometer la seguridad.
- Cumplimiento normativo: Aislamiento facilita cumplir con leyes como GDPR o HIPAA.

Ejemplo: Imaginar un edificio de departamentos:

- El edificio es Azure (una sola infraestructura).
- Cada departamento es un tenant.
- Todos usan los mismos servicios (agua, luz, internet), pero:
- Cada quien tiene su llave (autenticación).
- Cada quien paga lo que consume (modelo de pago por uso).
- Nadie puede entrar al departamento del otro (aislamiento).

Relación con características del cómputo en la nube

Característica	Relación con multitenencia / aislamiento
Elasticidad	Los tenants pueden escalar sin afectar a otros.
Acceso bajo demanda	Cada tenant usa servicios según necesidad.
Seguridad y aislamiento	Protege datos y servicios de cada cliente.
Modelo de autoservicio	Cada organización gestiona sus recursos.

1.2.5. Alta Disponibilidad y Tolerancia a Fallos

Es la capacidad de un sistema para estar operativo y accesible la mayor parte del tiempo, incluso si hay fallos en algunos de sus componentes.

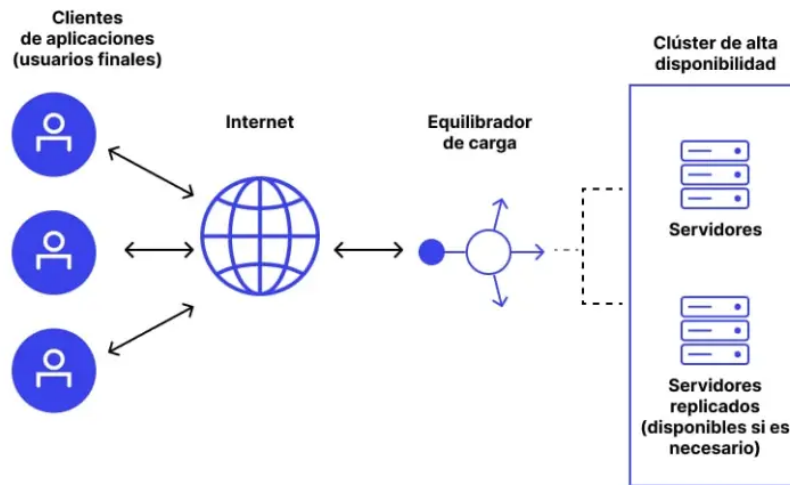
Objetivo:

- Minimizar el tiempo de inactividad.
- Garantizar la continuidad del servicio ante fallos menores o mantenimiento.

En la nube se lleva acabo mediante:

- La replicación de servidores en varias zonas de disponibilidad (AZ).
- Utilizando balanceadores de carga (Load Balancers).
- Con la recuperación automática de servicios.

- Con redundancia geográfica en regiones distintas.



Ejemplo en Azure:

Un servicio web desplegado en dos zonas de disponibilidad con un Azure Load Balancer delante. Si un centro de datos falla, el tráfico se redirige al otro automáticamente.

Tolerancia a Fallos (Fault Tolerance): Es la capacidad de un sistema para seguir funcionando correctamente incluso cuando uno o más de sus componentes fallan.

Objetivo:

- Que el sistema no se detenga en absoluto, aunque haya errores.
- Ofrecer resiliencia total frente a fallos críticos.

Se implementa en la nube mediante:

- El uso de sistemas distribuidos con réplicas activas.
- Duplicando los servicios en diferentes regiones y zonas.
- Aplicando estrategias como RAID, clusters, backups automáticos, etc.

Ejemplo en Azure:

Una base de datos distribuida (como Cosmos DB) que mantiene múltiples réplicas activas en distintas regiones. Si una región entera falla, la base de datos sigue operando desde otra sin pérdida de datos.

Diferencia entre Alta Disponibilidad y Tolerancia a Fallos

Característica	Alta Disponibilidad (HA)	Tolerancia a Fallos (FT)
Objetivo principal	Minimizar el tiempo fuera de servicio	Continuar operando sin interrupciones
Permite cierto fallo	Sí, pero puede haber una breve interrupción	No, el sistema continúa sin interrupción perceptible
Costos	Moderados	Más altos (más redundancia y complejidad)
Ejemplo común	Web App con múltiples instancias y balanceador	Base de datos distribuida con réplicas activas
Resiliencia ante fallos graves	Parcial	Completa (diseñada para soportar fallos críticos)

Relación con características del cómputo en la nube

Característica	Relación con HA / FT
Elasticidad	Permite agregar instancias para mantener disponibilidad.
Recuperación ante desastres	FT contribuye directamente con respaldo activo.
Escalabilidad	Compatible con escalado horizontal.
Alta disponibilidad como servicio	Azure, AWS y GCP lo ofrecen como parte de sus SLA.

1.2.6. Automatización y Autoservicio

La automatización en la nube se refiere al uso de herramientas y servicios que permiten realizar tareas sin intervención humana de forma repetitiva, precisa y rápida.

Las tareas que se pueden realizar son:

- Despliegue de aplicaciones
- Configuración de infraestructura (IaC - Infrastructure as Code)
- Copias de seguridad
- Actualizaciones de software
- Escalado automático de recursos
- Monitoreo y alertas



Ejemplo, Azure Resource Manager (ARM), Azure DevOps y PowerShell permiten automatizar el aprovisionamiento de máquinas virtuales, redes, bases de datos, etc.

El Autoservicio bajo demanda es la capacidad que tienen los usuarios de provisionar recursos informáticos automáticamente (como servidores, almacenamiento o redes) sin necesidad de contactar a personal de TI.

Se aplica en la nube a través de portales web, interfaces de línea de comandos (CLI) o APIs. El usuario elige qué necesita (ej. una VM con Ubuntu y 8 GB RAM) y el proveedor (como Azure o AWS) lo crea al instante.

En Azure, un estudiante puede iniciar sesión en el portal de Azure, crear una máquina virtual desde cero y comenzar a trabajar sin intervención del área de sistemas.

Diferencias y relación

Concepto	Definición breve	¿Interviene el usuario?
Automatización	Tareas que se ejecutan automáticamente según reglas	No (o solo una vez para configurarla)
Autoservicio	El usuario crea y configura recursos cuando los necesita	Sí (pero sin pedir ayuda a terceros)

Ambos trabajan juntos: el autoservicio lanza tareas automatizadas para crear recursos, configurar redes, instalar software, etc.

Relación con características del cómputo en la nube

Característica	Aplicación concreta
Autoservicio bajo demanda	El usuario solicita recursos cuando los necesita
Elasticidad	Se crean (o eliminan) recursos automáticamente
Escalabilidad automática	Automatización permite añadir/quitar recursos según demanda
Optimización de costos y tiempo	Reduce tareas manuales repetitivas

1.2.7. Seguridad y Cumplimiento Normativo

Concepto: Controles de seguridad integrados para proteger datos y cumplir con regulaciones.

Ejemplo: Azure ofrece servicios como Azure Security Center y Azure Active Directory para proteger datos sensibles y cumplir con normas como GDPR y HIPAA.

Comprender los riesgos, buenas prácticas y marcos regulatorios relacionados con la seguridad en entornos de nube dado que los datos se almacenan fuera del entorno físico de la empresa y existen brechas de seguridad que afectan la privacidad, la reputación y el cumplimiento legal.

Los principales riesgos de seguridad pueden ser:

Riesgo	Descripción
Pérdida de datos	Borrado accidental o ataque externo que elimina información crítica.
Fugas de información	Acceso no autorizado a datos sensibles.
Amenazas internas	Personal con acceso indebido o que actúa con malas intenciones.
Falta de control	Delegar infraestructura puede reducir el control directo sobre la seguridad.
Vulnerabilidades compartidas	Riesgos en ambientes multiusuario (multi-tenant) si hay fallas de aislamiento.

Entre las principales medidas de seguridad se encuentran:

A. La seguridad técnica

- Cifrado de datos (en tránsito y en reposo).
- Firewalls y detección de intrusos.
- Gestión de identidades y accesos (IAM).
- Autenticación multifactor (MFA).
- Copias de seguridad automatizadas.

B. La seguridad Administrativa

- Políticas de acceso mínimo necesario.

- Control de cambios y auditoría.
- Capacitación de usuarios.

C. La seguridad Física

- Centros de datos con vigilancia, controles biométricos, respaldo eléctrico, etc.

El cumplimiento normativo incluye normas y estándares clave:

Norma/Estándar	Enfoque
ISO/IEC 27001	Gestión de seguridad de la información.
GDPR (UE)	Protección de datos personales.
HIPAA (EE.UU.)	Privacidad en datos de salud.
SOC 2	Controles de seguridad, disponibilidad, confidencialidad, etc.
Ley Federal de Protección de Datos Personales (México)	Protección de datos en posesión de particulares.

La responsabilidad compartida (Shared Responsibility Model)

En la nube, la seguridad es una responsabilidad compartida entre el proveedor de servicios (ej. AWS, Azure, GCP) y el cliente.

Elemento	Proveedor	Cliente
Infraestructura física	✓	✗
Virtualización, hardware	✓	✗
Sistema operativo	✗	✓
Aplicaciones y datos	✗	✓

1.2.8 Buenas Prácticas para Cumplimiento

- Elegir proveedores certificados (ISO 27001, SOC 2, etc.).
- Firmar acuerdos de nivel de servicio (SLA) que incluyan seguridad.
- Realizar auditorías periódicas.

- Documentar procedimientos y controles.
- Mantenerse actualizado en legislación vigente.

La seguridad en la nube no es opcional, es una parte integral de cualquier arquitectura moderna. Cumplir con normativas no solo evita sanciones, también genera confianza con clientes y usuarios y el enfoque proactivo y la educación continua son claves para minimizar riesgos.

1.2.9. Movilidad y Flexibilidad

Concepto: Capacidad para cambiar rápidamente de proveedor o entorno sin grandes costos.

Ejemplo: Docker permite a los desarrolladores crear aplicaciones en contenedores que pueden ser ejecutados en cualquier entorno compatible, como AWS, Azure o Google Cloud, sin tener que reescribir el código.

Docker es una plataforma de código abierto que facilita el desarrollo, despliegue y ejecución de aplicaciones en contenedores. Los contenedores son unidades estandarizadas que incluyen todo lo necesario para que una aplicación se ejecute, como código, bibliotecas y dependencias, lo que asegura que la aplicación funcione igual en diferentes entornos.